

Infrastructure for the Agentic Web: Gap Analysis and Architecture from the Agentverse Platform

Robin Dey

OpenHub Research

Chiang Mai, Thailand

<https://github.com/web3guru888/agentverse-2030-paper>

ROBIN@OPENHUBRESEARCH.ORG

Panyanon Viradecha

OpenHub Research

Chiang Mai, Thailand

Abstract

The emergence of autonomous AI agents as first-class participants in digital infrastructure marks a fundamental inflection point in the evolution of the Web. While significant research effort has been directed at the behaviour and reasoning of individual agents, comparatively little attention has been paid to the *infrastructure* those agents require to operate reliably at scale—the protocols, services, and platforms that mediate their interactions with each other and with the world. This paper addresses that gap by presenting a systematic analysis of Agentverse, the agent cloud platform developed by Fetch.ai and operated under the Artificial Superintelligence (ASI) Alliance—currently comprising Fetch.ai, SingularityNET, and CUDOS—which represents one of the most mature and complete production deployments of agent-native infrastructure available today.

We make three principal contributions. First, we conduct a rigorous empirical audit of the Agentverse platform, cataloguing its 204 documented API endpoints and characterising what is operational, what is partially deployed, and what remains absent. From this audit we derive a structured **Gap Taxonomy**—eight categories encompassing approximately fifty missing infrastructure capabilities, ranging from agent memory and observability to security, economic primitives, and enterprise-grade scaling. Second, we propose a **seven-layer Agent Cloud Stack**—a principled reference architecture for what a fully realised agent-native cloud platform should provide by 2030, drawing on analogies from the evolution of general-purpose cloud computing and grounded in the specific gaps we identify. Third, we characterise five **critical evolution paths** along which Agentverse and platforms like it must travel: from ephemeral key-value storage to a full Agent Memory Cloud; from keyword-based agent discovery to a semantic, trust-weighted Agent DNS; from a single-protocol communication model to a multi-standard agent lingua franca; from single-instance hosting to agent-native orchestration at Kubernetes scale; and from simple token payments to a rich ecosystem of agent economic primitives.

Taken together, these contributions provide both a diagnostic of the current state of agent infrastructure and a concrete, technically grounded vision for what the agent cloud must become to support the agentic web—what an emerging body of literature terms Web4—by 2030.

Keywords: AI agent infrastructure, Agentverse, ASI Alliance, agent cloud, agent memory, A2A protocol, Model Context Protocol, Web4, decentralised AI, agentic web, gap analysis, reference architecture

1 Introduction

The history of the World Wide Web can be read as a succession of enabling infrastructures. Web1 required HTTP servers and DNS to make static documents universally accessible. Web2 required elastic cloud compute—Amazon Web Services, Google Cloud Platform, Microsoft Azure—to make dynamic, user-generated applications viable at global scale. Web3 required programmable blockchains to introduce verifiable ownership and trustless coordination into the web’s fabric. Each transition was enabled not primarily by new *applications* but by new *infrastructure*: foundational platforms that made a previously impractical class of systems tractable.

We are now at the beginning of a fourth such transition. Web4, variously termed the *agentic web* or the *symbiotic web* (EPH4 and WhatIsWeb4, 2026), is characterised by the presence of autonomous AI agents as primary actors on the internet—software entities that perceive their environment, reason about goals, execute multi-step plans, communicate with other agents, and transact economically, all without requiring human intervention at each step. The transition from Web3 to Web4 parallels in important respects the transition from Web1 to Web2: the applications (agents) already exist in prototype form, but the infrastructure required to operate them reliably, safely, and at scale remains nascent.

Chan et al. (2025) articulate the concept of *agent infrastructure*—“technical systems and shared protocols external to agents that are designed to mediate and influence their interactions with and impacts on their environments”—and argue that such infrastructure will be as foundational to agent ecosystems as the internet protocols are to the web. Their work, published in the *Transactions on Machine Learning Research*, provides an important theoretical taxonomy but stops short of grounding it in a specific production platform, or of specifying the concrete services and APIs that such infrastructure should expose. This paper takes that next step.

We study **Agentverse** (agentverse.ai), the agent cloud platform developed by Fetch.ai and now operated under the **Artificial Superintelligence (ASI) Alliance**—currently comprising Fetch.ai, SingularityNET, and CUDOS.¹ Agentverse is a compelling case study for several reasons. It is, to our knowledge, the most feature-complete agent-native cloud platform currently in production, offering agent hosting, a global agent registry (the Almanac), asynchronous messaging (mailboxes), a payment protocol, a multi-model LLM API (ASI:One), and an MCP integration layer. It is built on a decentralised substrate (Fetch.ai’s blockchain and the forthcoming ASI Chain), giving it architectural properties—open identity, permissionless participation, cryptographic trust—that hyperscaler agent of-

1. The ASI Alliance was originally formed in June 2024 as a merger of Fetch.ai, SingularityNET, and Ocean Protocol (Fetch.ai, 2024). Ocean Protocol announced its withdrawal from the Alliance effective October 10, 2025 (Ocean Protocol Foundation, 2025), citing differences over tokenomics and governance. The Alliance now comprises Fetch.ai, SingularityNET, and CUDOS, and the ASI token (formerly FET) remains the unified currency of the ecosystem.

ferings (Amazon Web Services, 2025; Microsoft, 2025; Google, 2026) do not share. And crucially, it is a production system with real registered agents (over 36,000 in the Almanac as of Q1 2026 (Kantor, 2026)), real developer communities, and real API endpoints that can be empirically tested.

Our investigation proceeds from a hands-on audit of all 204 documented Agentverse API endpoints, supplemented by analysis of the uAgents SDK, the ASI:One LLM API, and the platform’s MCP integration layer. From this empirical foundation we derive our gap taxonomy, architecture, and evolution paths.

The remainder of the paper is structured as follows. Section 2 surveys related work on agent infrastructure, cloud computing evolution, the current landscape of agent platforms, and the emerging protocol standards that define how agents communicate. Section 3 characterises the current state of the Agentverse platform in detail. Section 4 presents our Gap Taxonomy. Section 5 proposes the seven-layer Agent Cloud Stack as a reference architecture for 2030. Section 6 discusses five critical evolution paths. Section 7 places Agentverse in competitive context and discusses open research problems. Section 8 concludes.

2 Background and Related Work

2.1 Agent Infrastructure

The theoretical foundations of agent infrastructure are established by Chan et al. (2025), who identify three core functions that such infrastructure must serve: *attribution* (establishing which agent, user, or organisation is responsible for a given action); *interaction shaping* (protocols and systems that constrain or coordinate agent behaviour); and *detection and remediation* (mechanisms for identifying and correcting harmful agent actions). Their work provides an important conceptual framework but does not address the engineering question of *what services* a practical agent cloud platform must expose to fulfil these functions.

Complementary work by Hussein et al. (2025) projects that the global population of AI agents will increase by more than $100\times$ between 2026 and 2036, potentially reaching trillions of instances. If accurate, this scale of deployment makes the question of agent infrastructure not merely an engineering convenience but a civilisational necessity: the protocols and platforms that govern how trillions of agents interact with each other and with humans will shape outcomes in commerce, science, governance, and beyond. Near-term evidence supports this trajectory: Kantor (2026) reports over 36,000 agents already registered on Agentverse as of March 2026, a figure that would have seemed implausible even two years prior.

The economics of agent interaction are explored by Hadfield and Koh (2025), who model the agent economy as a labour market in which AI agents substitute for human workers across increasingly complex task domains. Their analysis implies significant demand for infrastructure that supports agent *hiring*, *payment*, *trust evaluation*, and *performance guarantees*—categories that map directly to gaps we identify in Section 4.

2.2 Cloud Computing as a Reference Model

The evolution of general-purpose cloud computing provides a useful reference model for understanding what agent cloud infrastructure should become. AWS launched in 2006 with three primitive services: S3 (object storage), EC2 (compute), and SQS (messaging). Over the following twenty years, it expanded to over 200 services spanning databases, networking, security, observability, machine learning, and developer tooling. The key insight from this trajectory is that *primitive compute and storage are table stakes*; the services that create lasting competitive advantage are those that solve the hard operational problems developers face *after* they have compute— authentication, persistence, observability, scaling, and compliance.

We observe that Agentverse in 2026 is approximately at the AWS 2006 stage: it has primitive compute (hosted agents), a registry (Almanac), and messaging (mailbox). The gap between this and a mature agent cloud platform is analogous to the gap between AWS in 2006 and AWS today.

The cloud computing literature establishes a layered abstraction model: IAAS $\rightarrow$ PAAS $\rightarrow$ SAAS $\rightarrow$ FAAS. We propose extending this model with a new layer: **AaaS (Agents as a Service)**, characterised by persistent identity, autonomous execution, economic agency, and protocol-mediated interoperability. The seven-layer Agent Cloud Stack we propose in Section 5 can be understood as the AaaS equivalent of the OSI networking model: a principled decomposition of the full stack that any AaaS platform must eventually provide.

2.3 Current Agent Platform Landscape

The agent platform landscape in 2026 divides roughly into two camps: hyperscaler offerings from established cloud providers, and decentralised agent networks built on blockchain substrates.

Hyperscaler platforms—Amazon Bedrock AgentCore (generally available October 13, 2025) ([Amazon Web Services, 2025](#)), Azure AI Foundry with Microsoft Entra Agent ID (first announced May 2025, public preview at Microsoft Ignite November 2025, with governance extensions at RSAC 2026) ([Microsoft, 2025](#)), and the Gemini Enterprise Agent Platform (announced at Google Cloud Next, April 22, 2026, consolidating Vertex AI and AgentSpace under a unified product) ([Google, 2026](#))—offer mature compute, storage, and identity infrastructure that agents can leverage, but were designed primarily for centralised enterprise deployments. They inherit the full managed-service richness of their parent cloud platforms (databases, queues, caches, identity, compliance), but their agent identity systems are proprietary, their discovery mechanisms are siloed, and their economic models reduce to cloud billing rather than agent-native micropayments. Critically, they lack the open, permissionless participation model that a truly interoperable agentic web requires.

Decentralised agent networks—Agentverse (Fetch.ai / ASI Alliance), Virtuals Protocol, and ElizaOS—are built on open protocols and cryptographic identity, enabling permissionless agent registration and cross-platform interoperability. They are closer in spirit to the internet itself (open, decentralised, protocol-governed) than to AWS (centralised, propri-

etary, subscription-governed). However, they currently lack the managed service richness of hyperscaler platforms, which is precisely the gap this paper analyses.

Table 9 summarises the current capabilities of leading agent platforms along dimensions we consider essential for a production agent cloud. We return to this comparison in Section 7.

2.4 Protocol Standards Landscape

Three protocol standards are converging to define the communication layer of the agentic web.

The **Model Context Protocol (mcp)** (Anthropic, 2025; MCP Core Maintainers, 2026), with its latest specification dated November 25, 2025, defines how AI agents connect to external tools, data sources, and services. Originally conceived as a client-server protocol for LLM tool use, MCP has evolved into a general-purpose context provision standard supporting distributed execution, stateless transport (addressing early load-balancing limitations), and a security framework. The MCP specification uses date-based versioning rather than semantic versions; the ecosystem has expanded to over 110 million monthly downloads (MCP Core Maintainers, 2026), signalling broad industry adoption.

The **Agent-to-Agent (A2A) protocol** (Linux Foundation and Google, 2026), released at v1.0 in April 2026 and now supported by over 150 organisations, defines how AI agents discover and interact as peers—sharing tasks, streaming results, and coordinating work across organisational and framework boundaries. Donated by Google to the Linux Foundation, A2A has been integrated into AWS, Microsoft, and Google cloud platforms, positioning it as the de facto standard for inter-agent communication in enterprise contexts. The Linux Foundation founding partners include Amazon Web Services, Cisco, Google, Microsoft, Salesforce, SAP, and ServiceNow.

The **Fetch.ai Agent Chat Protocol**—with digest `proto:30a801ed...d33d62`²—predates both MCP and A2A and is the native communication standard of the Agentverse ecosystem. It defines `ChatMessage`, `ChatAcknowledgement`, and content types including `TextContent`, `StartSessionContent`, `EndSessionContent`, and `ResourceContent`. While less widely adopted outside the Fetch.ai ecosystem, it offers a complete interaction model including session management and resource sharing.

A central thesis of this paper is that by 2030, these three standards will have converged—or that a platform like Agentverse will serve as a multi-protocol gateway bridging all three—in the same way that TCP/IP emerged from the competing network protocol landscape of the 1980s.

2.5 Agent Memory

A body of rapidly growing research addresses the problem of *agent memory*—how agents maintain persistent context, accumulate knowledge, and improve over time. Mem0 (2026) identify 21 agent frameworks with memory support and three distinct hosting models: managed cloud, open-source self-hosted, and local MCP. Graph-based memory systems such as

2. Full digest: `proto:30a801ed3a83f9a0ff0a9f1e6fe958cb91da1fc2218b153df7b6cbf87bd33d62`

Zep’s Graphiti (Zep AI, 2025) represent the current state of the art, achieving 63.8% accuracy on LongMemEval benchmarks versus 49.0% for flat vector store approaches.

Despite this active research landscape, no major agent cloud platform currently offers hosted, authenticated, per-agent memory as a managed service—equivalent to what RDS or DynamoDB provides for web applications. This gap, which we term the *Memory Cloud Gap*, is arguably the single most consequential missing capability in current agent infrastructure, and we develop it in detail in Sections 4 and 6.

2.6 Agent Identity and Trust

Work on decentralised identity for AI agents (Garzon et al., 2025) proposes combining W3C Decentralised Identifiers (DIDs) with Verifiable Credentials (VCs) to enable agents to prove ownership and capabilities across platform boundaries. Microsoft’s Entra Agent ID (Microsoft, 2025) and dedicated agent trust frameworks such as MolTrust (MolTrust, 2026) signal that agent identity is becoming an active area of enterprise investment. However, the question of *portable* agent identity—whether an agent created on Agentverse can prove its identity, reputation, and capabilities on AWS—remains unresolved, and we discuss it as a key open problem in Section 7.

2.7 Agent Safety and Governance

The OWASP Agentic AI Security Top 10, published in December 2025, provides the first formal taxonomy of risks specific to autonomous AI agents, covering goal hijacking, tool misuse, prompt injection, cascading failures, and identity abuse (OWASP Foundation, 2025). Microsoft’s Agent Governance Toolkit (open-sourced April 2026) maps these risks to capability sandboxing, semantic intent classification, and MCP security gateways (Microsoft Open Source, 2026). Bessemer Venture Partners (2026) characterise agent security as “the defining cybersecurity challenge of 2026”. Despite this, current agent cloud platforms—including Agentverse—offer minimal built-in governance controls, a gap we document systematically in Section 4.

3 The Agentverse Platform: Current State

3.1 Platform Overview

Agentverse (agentverse.ai) is the agent cloud platform of the Artificial Superintelligence (ASI) Alliance, currently comprising Fetch.ai, SingularityNET, and CUDOS (ASI Alliance, 2026). It provides a vertically integrated stack for deploying, discovering, and operating autonomous AI agents on top of the Fetch.ai blockchain and the forthcoming ASI Chain mainnet (TestNet phase as of Q2 2026, mainnet targeted for late 2026).

The platform consists of six primary components:

1. **Agent Hosting:** A managed execution environment for Python agents written using the uAgents SDK. Developers upload code via API, start and stop agents programmatically, and retrieve execution logs. Each agent is assigned a cryptographically derived address (agent1q...) and allocated a computation quota measured in execution seconds.

2. **The Almanac:** A global agent registry mapping agent addresses to their endpoints, supported protocols, and capability metadata. The Almanac is publicly readable and provides both exact-address lookup and recently-registered agent feeds. As of Q1 2026, over 36,000 agents are registered in the Almanac (Kantor, 2026), reflecting the rapid growth of the developer ecosystem.
3. **Search:** A keyword and semantic search interface over registered agents, supporting protocol-digest-based filtering to find agents that implement specific interaction schemas.
4. **Mailbox:** An asynchronous message buffer that holds incoming messages for agents that are temporarily offline, enabling reliable agent-to-agent communication without requiring both parties to be simultaneously active.
5. **ASI:One:** A multi-model LLM API offering seven models via an OpenAI-compatible endpoint (`api.asi1.ai`): `asi1`, `asi1-fast`, `asi1-extended`, `asi1-graph`, `asi1-agentic`, `asi1-fast-agentic`, and `asi1-extended-agentic`. The API additionally supports real-time web search, structured output with JSON schema, image generation, and streaming.
6. **Payment Protocol:** A built-in buyer-seller protocol supporting Stripe (fiat), Skyfire USDC, and FET/ASI on-chain payments, enabling agents to charge for services and pay each other autonomously (Fetch.ai, 2026).

The uAgents SDK (Python) provides the developer-facing abstraction: agents are defined as Python modules that respond to lifecycle events (`on_event("startup")`), incoming messages (`@protocol.on_message`), and REST requests (`@agent.on_rest_get`, `@agent.on_rest_post`). Persistent state is available via `ctx.storage` (a key-value store). Agent-to-agent communication uses the Agent Chat Protocol with typed message schemas.

An MCP integration layer is available at two endpoints—a full endpoint (`mcp.agentverse.ai/sse`) and a lightweight one (`mcp-lite.agentverse.ai/mcp`)—exposing platform capabilities to MCP-compatible clients such as Claude Desktop and FetchCoder.

3.2 API Audit Methodology

Our gap analysis is grounded in a systematic empirical audit of the Agentverse API surface. The platform exposes approximately **204 documented endpoints** across two API versions (v1 and v2) at <https://agentverse.ai>. We tested each endpoint category with authenticated requests using a production API key (scope: `av`), supplemented by analysis of the Fern-generated documentation, SDK source code, and community resources.

Key findings from the audit:

- **29 endpoints are fully operational** with the standard bearer token.
- **15 endpoints require specific authentication or parameters** beyond the basic bearer token (notably, mailbox endpoints require agent-level attestation via challenge-response).
- **V2 Almanac and most V2 Hosting endpoints return 404**—they are documented but not yet deployed to production.

- **The practical working api mixes V2 for agent crud** (create, read, update, delete) **and V1 for hosting, search, storage, and almanac operations.**
- **v1beta1 is fully decommissioned**—all endpoints return 404.
- Storage is limited to a key-value interface (`ctx.storage`) accessible via `GET /v1/hosting/agents/{address}/storage`; there is no structured query capability, no indexing, and no size visibility.
- Secrets management is account-scoped (not per-agent), creating security isolation concerns for multi-agent deployments.
- Log retrieval is polling-only via `GET /v1/hosting/agents/{address}/logs/latest`; there is no streaming, no historical log retention beyond a rolling window, and no log search.

This audit provides the empirical foundation for the gap taxonomy presented in Section 4. The discrepancy between documented and deployed endpoints is itself instructive: it suggests that the platform team has articulated a broader vision than the current production system realises, consistent with our characterisation of Agentverse as an early-stage agent cloud undergoing rapid expansion.

3.3 What Currently Works Well

Before proceeding to the gap analysis, we note the capabilities that Agentverse provides competently and that distinguish it from competing platforms:

Cryptographic agent identity. Every agent has a provable identity derived from a key pair, enabling trustless attribution of actions without a central authority. This is architecturally superior to IAM-based identity systems that depend on proprietary infrastructure.

Open protocol communication. The Agent Chat Protocol provides a typed, schema-validated communication standard that any agent can implement, regardless of the underlying framework or language. The protocol digest functions as a capability advertisement: agents that share a protocol digest can communicate without custom integration.

Permissionless registration. Any developer can register an agent in the Almanac without approval, enabling a genuinely open ecosystem. The over 36,000 agents discoverable in the Almanac as of Q1 2026 reflects the traction this openness creates.

Multi-model llm access. ASI:One provides access to seven specialised models via a single API, including an agentic model family designed for tool use and multi-step reasoning, and a graph model for structured knowledge tasks.

Native payment protocol. The integration of payment as a first-class protocol (rather than a bolt-on billing system) reflects a principled design choice: agents should be able to charge for and pay for services as a native capability, not via out-of-band invoicing.

Decentralised substrate. Unlike hyperscaler agent platforms, Agentverse is built on an open blockchain, meaning that the Almanac, agent identities, and on-chain transactions are not owned by any single corporation and cannot be unilaterally revoked.

These strengths position Agentverse as the correct foundation on which to build the agent cloud of 2030. The gap analysis that follows should be read not as a critique but as a roadmap.

4 Gap Analysis: What the Agent Cloud Is Missing

We organise the identified gaps into eight categories, each corresponding to a layer of the agent cloud stack we propose in Section 5. For each gap we note its cloud computing analogue (the equivalent service that general-purpose cloud platforms provide), a severity assessment (High / Medium / Low, based on how significantly the absence impairs production agent deployments), and—where applicable—the specific API evidence from our audit.

4.1 Category A: Agent Memory and State

Agentverse provides `ctx.storage`—a simple key-value store accessible within agent code via `await ctx.storage.get(key)` and `await ctx.storage.set(key, value)`. This is the *only* persistent state mechanism available to hosted agents. It corresponds roughly to writing to a flat file: there is no schema, no query capability, no indexing, no size quota visibility, and no sharing between agents.

Contemporary agent memory research (Mem0, 2026; Zep AI, 2025) distinguishes four cognitively motivated memory types: *episodic* (what happened, when, with whom), *semantic* (facts, concepts, relationships), *procedural* (how to do things, learned workflows), and *working* (the active context of a current task). A mature agent cloud must provide hosted, authenticated services for each type. The analogy to general-purpose cloud is precise: `ctx.storage` is to agent memory what writing to a local disk is to web application data—it works for a single instance in development but fails in production at scale.

The absence of memory-as-a-service has a compounding effect on agent capability. Without persistent semantic memory, every agent interaction begins from zero context. Without shared memory spaces, multi-agent workflows cannot maintain coherent state. Without episodic memory, agents cannot learn from experience across sessions. We argue in Section 6 that closing this gap—by providing hosted, authenticated MCP servers for each memory type—is the highest-priority single improvement Agentverse could make to agent capability.

4.2 Category B: Agent Observability

Debugging and monitoring a hosted agent on Agentverse today requires polling a rolling-window log endpoint and scanning the output for `ctx.logger.info()` calls. There is no structured log search, no trace visualisation, no cost tracking, and no performance metrics. This stands in stark contrast to the rich observability ecosystems that have emerged for LLM applications—LangSmith (LangChain, 2026), AgentOps (AgentOps, 2026), and Langfuse (Langfuse, 2026)—all of which offer distributed tracing, token cost tracking, and behaviour analytics.

Table 1: Category A gaps: Agent Memory and State

Gap	Description	Cloud logue	Ana-	Severity
No vector store	No embedding-based semantic similarity search	OpenSearch Pinecone	/	High
No knowledge graph	No relationship-structured fact storage with temporal validity	Neptune / Neo4j		High
No episodic memory	No conversation history with temporal indexing	DynamoDB with TTL		High
No shared memory spaces	No permissioned shared context between cooperating agents	ElastiCache		High
No memory MCP servers	No per-agent authenticated MCP endpoints for memory access	—		High
<code>ctx.storage</code> opacity	No visible quota, no overflow handling	—		Medium
No procedural memory	No storage for learned workflows / tool usage patterns	Step state	Functions	Medium

Table 2: Category B gaps: Agent Observability

Gap	Description	Cloud logue	Ana-	Severity
No distributed tracing	No OpenTelemetry-compatible trace of execution paths	CloudWatch X-Ray		High
No cost tracking	No per-agent LLM token usage or compute cost visibility	Cost Explorer		High
No streaming logs	Logs available only via polling, not streaming	CloudWatch Logs Insights		High
No historical log retention	Rolling window only; no searchable history	S3 log archival		Medium
No performance metrics	No latency, throughput, or error rate dashboards	CloudWatch Metrics		Medium
No behaviour analytics	No aggregate view of agent actions across interactions	Application insights	In-	Medium
No A/B testing framework	No infrastructure for comparing agent behaviour variants	Evidently		Low
No anomaly detection	No alerts on unexpected agent behaviour patterns	CloudWatch Anomaly		Medium

4.3 Category C: Agent Security and Governance

Agentverse’s security model is coarse-grained: a hosted agent either runs or it does not; there is no capability-based permission system governing what an agent may access, call, or modify. Account-level secrets (accessible via `GET /v1/hosting/secrets`) are shared across all agents under an account, creating a security isolation failure for multi-agent deployments. There is no sandboxed execution environment, no content filtering on inputs or outputs, and no audit trail of agent actions beyond the rolling log window.

This is a significant concern given the OWASP Agentic AI Security Top 10 ([OWASP Foundation, 2025](#)), which identifies prompt injection, tool misuse, goal hijacking, and cascading failures as the primary risk categories for autonomous agents. The absence of a platform-level policy engine means that developers must implement all safety controls within agent code—which is both error-prone and inconsistent.

Table 3: Category C gaps: Agent Security and Governance

Gap	Description	Cloud logue	Ana-	Severity
No capability per- missions	Agents have undifferentiated access to all APIs	IAM	least-	High
No sandbox isola- tion	No Wasm or container-level isolation between agents	Firecracker	mi-	High
No per-agent se- crets	Secrets are account-scoped, not agent-scoped	Secrets Manager		High
No policy engine	No declarative rules constraining agent actions	Org SCPs		High
No content filtering	No input/output guardrails for harmful content	Bedrock		High
No audit trail	No immutable record of agent actions beyond rolling logs	Guardrails		Medium
No rate limiting per agent	Rate limits apply at account level only	CloudTrail		Medium
No action attribu- tion	No cryptographic proof linking action to agent	API plans	Gateway	Medium
		—		Medium

4.4 Category D: Agent Networking and Communication

The Agentverse communication model is built around the Agent Chat Protocol: an agent sends a `ChatMessage` to another agent’s address, the platform routes it via the Almanac, and the recipient responds. This model works well for synchronous request-response interactions but is insufficient for the coordination patterns that production multi-agent systems require.

Notably absent is native support for the A2A protocol ([Linux Foundation and Google, 2026](#)), which as of April 2026 has been adopted by over 150 organisations and integrated into AWS, Azure, and Google Cloud. An Agentverse agent cannot natively participate in

an A2A task graph—it must use an adapter (`uagents-adapter[a2a-inbound/outbound]`) that translates between protocols. As A2A becomes the de facto enterprise standard, this gap risks positioning Agentverse as an isolated ecosystem rather than a participant in the broader agentic web.

Table 4: Category D gaps: Agent Networking and Communication

Gap	Description	Cloud logue	Ana-	Severity
No native A2A support	Cannot natively participate in A2A task graphs	—		High
No pub/sub event bus	No broadcast coordination between multiple agents	SNS / EventBridge		High
No streaming messages	Request-response only; no SSE or WebSocket	API Gateway WS		High
No group coordination	No consensus, voting, or delegation primitives	— (novel)		Medium
No service mesh	No load balancing across agent replicas	App Mesh		Medium
No push / web-hooks	No push notification to external systems	EventBridge		Medium
No human escalation	No standardised path for agent to request human input	—		Medium
No workflow orchestration	No native sequential/parallel agent task graph	Step Functions		High

4.5 Category E: Agent Development and Lifecycle

The development experience for Agentverse agents is constrained by the single-file paradigm: agent code is uploaded as a single `agent.py` file (encoded as a JSON string), with no support for multi-file projects, dependency declaration, or module imports beyond what the platform pre-installs. There is no version control for deployed code, no staging environment, no CI/CD integration, and no local development emulator—meaning that all testing must be done against production infrastructure.

4.6 Category F: Data and Integration Services

Beyond `ctx.storage`, Agentverse provides no managed data services. There is no hosted database (SQL or NoSQL), no object storage, no caching layer, and no API gateway for controlled external service access. Critically, there is no **mcp Server Hub**: a marketplace of pre-built, pre-authenticated MCP servers that agents can subscribe to. MCP has emerged as the dominant standard for connecting AI agents to external tools. A managed MCP server marketplace—offering agents access to web search, calendar, email, code execution, browser control, and database services—would dramatically expand the capability surface of every agent on the platform.

Table 5: Category E gaps: Agent Development and Lifecycle

Gap	Description	Cloud logue	Ana-	Severity
Single-file paradigm	No multi-file project support	Lambda	deploy-	High
No code versioning	No history of deployed code versions	CodeCommit / GitHub		High
No staging environment	No pre-production test environment	Lambda aliases		High
No local emulator	Cannot run hosted agent locally	SAM CLI		High
No CI/CD integration	No webhook-triggered deploy pipeline	CodePipeline		Medium
No dependency declaration	Pip packages limited to pre-installed set	Lambda layers		High
No testing framework	No unit or integration test harness for agents	—		Medium
No agent templates	No scaffolding beyond basic examples	SAM init		Low

Table 6: Category F gaps: Data and Integration Services

Gap	Description	Cloud logue	Ana-	Severity
No hosted database	No SQL or NoSQL managed database per agent	RDS / DynamoDB	Dy-	High
No object storage	No file / blob storage for agent assets	S3		High
No caching service	No in-memory cache for hot data	ElastiCache		Medium
No MCP Server Hub	No marketplace of authenticated MCP servers	AWS Marketplace		High
No API gateway	No controlled proxy for external API access	API Gateway		Medium
No task queue	No job queue for deferred or scheduled tasks	SQS		Medium
No data pipeline	No ETL / streaming ingestion for agent data	Kinesis / Glue		Low

4.7 Category G: Agent Economic Primitives

Agentverse includes a payment protocol and a basic marketplace, but lacks the richer economic infrastructure that a mature agent economy requires. The search API returns agent listings with interaction counts and star ratings, but there is no multi-dimensional trust score, no SLA enforcement mechanism, no automated pricing, and no revenue analytics for agent developers.

We use the term *agent economic primitives*—borrowing from mechanism design (Myerson, 2007)—to describe the set of basic operations that a functional agent economy requires: payment (exists), reputation (rudimentary), lending, insurance, and price discovery (all absent). Each has a direct precedent in human financial systems and is tractable to implement on a programmable blockchain.

Table 7: Category G gaps: Agent Economic Primitives

Gap	Description	Precedent	Severity
No multi-dim. reputation	Single star rating only	eBay / Stripe Risk	High
No SLA enforcement	No smart-contract-backed service guarantees	—	High
No revenue analytics	No earnings and transaction history dashboard	Stripe board Dash-	High
No automated pricing	No dynamic pricing based on demand or capability	Spot Instances	Medium
No resource lending	No borrowing of compute against future revenue	DeFi lending	Low
No agent insurance	No risk pooling for agent failures or errors	Lloyd’s syndicates	Low
No dispute resolution	No arbitration for failed service delivery	eBay Resolution	Medium
No KYA framework	No “Know Your Agent” compliance registry	KYC / AML	Medium

4.8 Category H: Scale and Enterprise

Agentverse currently supports a single execution instance per hosted agent. There is no horizontal scaling, no multi-region deployment, no private networking between agents, and no enterprise authentication integration. The platform has not published uptime SLAs, compliance certifications, or dedicated compute options. These gaps are significant for enterprise adoption in regulated industries—financial services, healthcare, legal—where guaranteed uptime, data residency, and compliance certifications are prerequisites.

Table 8: Category H gaps: Scale and Enterprise

Gap	Description	Cloud logue	Ana-	Severity
No horizontal scaling	Single instance per agent; no auto-scaling	ECS / Kubernetes		High
No multi-region	Single deployment region only	AWS multi-region		High
No private networking	All communication via public infrastructure	VPC / PrivateLink	Pri-	Medium
No SSO / enterprise IAM	No SAML, OIDC, or SCIM integration	Cognito / Okta		High
No published uptime SLA	No contractual uptime guarantee	AWS SLA		High
No compliance certifications	No SOC 2, ISO 27001, HIPAA	AWS Compliance		High
No dedicated compute	No reserved or dedicated execution tier	Reserved instances	In-	Medium
No data residency	No control over geographic data placement	AWS Regions		Medium

4.9 Summary

Across the eight categories, we identify **52 distinct capability gaps**. The concentration of High-severity gaps in Categories A (Memory), C (Security), D (Networking), E (Development), F (Data Services), and H (Enterprise) reflects the platform’s current positioning as a capable prototype rather than a production-grade infrastructure platform. Hyperscaler platforms close most Category F and H gaps by inheriting their parent cloud’s managed service catalogue, but lack Agentverse’s advantages in Categories A (agent-specific memory architecture), D (open communication protocol), and G (agent-native economic primitives). The architecture in Section 5 is designed to close the gaps while preserving these advantages.

5 A Layered Architecture for the Agent Cloud: 2030

Drawing on the gap analysis of Section 4 and the cloud computing reference model of Section 2.2, we propose a seven-layer reference architecture for a fully realised agent cloud platform. Each layer addresses one or more gap categories from Section 4. The layers are ordered from infrastructure (Layer 0) to application (Layer 6), following the convention of the OSI model.

5.1 Layer 0: Agent Substrate (Identity + Compute Foundation)

Identity substrate. Every agent should have a W3C Decentralised Identifier (DID) (Garzon et al., 2025; W3C, 2022) anchored to the ASI Chain. DIDs enable agents to prove identity across platform boundaries and carry Verifiable Credentials (VCs) attesting to capabilities,

Layer 6: Economy	-- Marketplace, reputation, KYA, economic primitives	
Layer 5: Observability	-- Tracing, cost dashboards, guardrail eval, sim env	
Layer 4: Services	-- AgentDB, AgentStore, AgentCache, AgentMCP Hub	
Layer 3: Communication	-- Multi-protocol gateway (Chat + A2A + MCP), event bus	
Layer 2: Memory	-- Episodic, Semantic, Procedural, Working, Shared (all via MCP)	
Layer 1: Runtime	-- Wasm sandbox, capability-based security, auto-scaling	
Layer 0: Substrate	-- ASI Chain, DIDs, VCs, TEEs	

Figure 1: The Seven-Layer Agent Cloud Stack. Layer 0 provides the decentralised trust substrate; Layers 1–6 add progressively higher-level services, each closing specific gaps identified in Section 4.

compliance certifications, and reputation. The existing `agent1q...` address format is conceptually compatible with DID anchoring; the ASI Chain provides the ledger needed for DID registration.

Execution anchoring. By 2030, agent execution records—start/stop events, resource consumption, output hashes—should be anchored on the ASI Chain, enabling permissionless verification of agent behaviour. Trusted Execution Environments (TEEs) should be available for agents handling sensitive data.

5.2 Layer 1: Agent Runtime (Execution Environment)

Sandboxed execution. Agent code should run in WebAssembly sandboxes with capability-based security. Each agent receives an explicit set of capability tokens at instantiation—access to specific MCP servers, specific secret names, specific network endpoints. Anything not explicitly granted is inaccessible. This directly addresses Category C gaps.

Multi-language support. The current Python-only restriction should be lifted. Wasm-compiled agents in TypeScript, Rust, and Go should be deployable alongside Python agents, enabling a broader developer community.

Auto-scaling. Popular agents should scale horizontally—multiple execution instances behind a load balancer—rather than being limited to a single instance. Auto-scaling requires that agent state be externalised (handled by Layer 2), making the memory and runtime layers interdependent.

5.3 Layer 2: Agent Memory (Persistence + Knowledge)

The memory layer is the most consequential addition to the current platform. We propose five services, each exposed as a per-agent authenticated MCP server:

Episodic Memory (`memory://agent/{address}/episodic`): Temporally indexed interaction histories—what happened, when, with whom, and with what outcome. Implemented as a time-series store with full-text and embedding-based retrieval.

Semantic Memory (`memory://agent/{address}/semantic`): A knowledge graph (Zep AI, 2025) storing facts, concepts, and relationships with temporal validity bounds. Supports multi-hop graph traversal and hybrid vector-graph retrieval.

Procedural Memory (`memory://agent/{address}/procedural`): Learned workflows, tool usage patterns, and action sequences that have proven effective.

Working Memory (`memory://agent/{address}/working`): A fast, short-lived cache for the active context of ongoing sessions, automatically evicted when the session ends.

Shared Memory Spaces (`memory://space/{space_id}`): Permissioned shared namespaces for multi-agent workflows. Access governed by Verifiable Credentials (Layer 0).

The critical architectural choice is that **all memory services are exposed as mcp servers**. This means any MCP-compatible agent or client can access its memory using

standard tool calls—no Agentverse-specific SDK required. Memory becomes a platform utility with a universal interface.

5.4 Layer 3: Agent Communication (Protocols + Networking)

Multi-protocol gateway. A gateway that transparently bridges the Agent Chat Protocol, A2A ([Linux Foundation and Google, 2026](#)), and MCP ([Anthropic, 2025](#)), enabling Agentverse agents to participate in A2A task graphs and consume MCP services without leaving the platform. This addresses the most strategically significant Category D gap.

Event bus. A pub/sub system—the agent-native equivalent of Amazon SNS or Google Pub/Sub—enabling broadcast coordination between agents and reactive, event-driven architectures.

Streaming support. Messages should support Server-Sent Events and WebSocket streaming for agents producing long-running outputs.

Group coordination primitives. Native support for consensus, delegation, and quorum patterns, enabling autonomous multi-agent organisations.

Human escalation protocol. A standardised, auditable path for an agent to request human oversight, satisfying EU AI Act requirements for high-risk systems ([OWASP Foundation, 2025](#)).

5.5 Layer 4: Agent Services (Managed Cloud Services)

The managed service catalogue that closes Category F gaps:

- **AgentDB:** Hosted relational and document database, provisioned per agent
- **AgentStore:** Object storage for agent-generated assets (S3 equivalent)
- **AgentCache:** Managed Redis-compatible in-memory cache
- **AgentQueue:** Durable task queue for deferred work and backpressure
- **AgentSecrets:** Per-agent secrets management with automatic rotation
- **AgentMCP Hub:** Marketplace of pre-built authenticated MCP servers—web search, calendar, email, code execution, browser control, financial data—subscribable with a single API call

5.6 Layer 5: Agent Observability (Monitoring + Debugging)

- **Distributed tracing:** OpenTelemetry-native spans for every LLM call, tool invocation, memory access, and message, aggregatable into execution trees
- **Cost dashboards:** Per-agent visibility into token consumption, compute seconds, storage, and payment flows
- **Behaviour analytics:** Aggregate views of interaction patterns, tool usage, and error clustering

- **Guardrail evaluation:** Automated testing of agent outputs against safety and quality criteria
- **Simulation environment:** Sandboxed testing against synthetic workloads and adversarial inputs

5.7 Layer 6: Agent Economy (Marketplace + Economic Primitives)

Semantic Agent Discovery. Natural-language capability search returning agents ranked by capability match, reputation, price, and current availability—the Almanac evolved from a phone book into a recommendation engine.

Multi-dimensional reputation. Composable on-chain reputation scores covering reliability, quality, safety, and speed—portable across any platform supporting the DID/VC trust model (Garzon et al., 2025).

SLA enforcement via smart contracts. Service Level Agreements automatically enforced by ASI Chain smart contracts, transforming the marketplace from trust-based to trust-verified.

Agent economic primitives (Myerson, 2007):

- *Resource lending:* Agents borrow compute capacity against future revenue
- *Risk pooling:* Developers insure against agent failures via shared reserves
- *Price discovery:* Dynamic rate negotiation based on demand, supply, and reputation history

Know Your Agent (KYA) compliance registry. Verifiable attestations of operator identity, intended use, safety certifications, and compliance status—the agent equivalent of KYC (ChainUp, 2026).

6 Critical Evolution Paths

We identify five specific transitions that Agentverse must undergo between its current state and the 2030 architecture of Section 5.

6.1 From ctx.storage to Agent Memory Cloud

Current state. `ctx.storage`—a flat key-value store with no schema, querying, indexing, size visibility, or cross-agent sharing.

2030 state. A four-tier memory architecture (episodic, semantic, procedural, working), plus shared memory spaces, all exposed as per-agent authenticated MCP servers.

The database analogy. This transition is equivalent to the shift from writing application data to local flat files to using a managed relational database. Before hosted databases, every web application implemented its own persistence. Hosted databases (MySQL, then RDS, then DynamoDB) transformed what was possible at acceptable development cost. Today every Agentverse agent that needs memory must implement its own persistence

outside the platform; an Agent Memory Cloud would make sophisticated memory available as a platform utility.

The MCP interface. Memory services exposed as MCP servers means any MCP-compatible framework can give its agents persistent memory by subscribing to the appropriate endpoint. Memory becomes infrastructure-level, not application-level. Recent work by [Zep AI \(2025\)](#) on graph-based memory achieves 63.8% accuracy on LongMemEval versus 49.0% for flat vector stores—a 30% relative improvement that would be immediately available to all Agentverse agents if integrated at the memory layer.

6.2 From Almanac to Semantic Agent DNS

Current state. The Almanac maps addresses to endpoints and protocols. Search supports keyword and rudimentary semantic matching over the registered agent population.

2030 state. A semantic, trust-weighted discovery system with real-time capability negotiation: a DNS plus search engine for the agent web.

The analogy. The early internet had WHOIS (address-to-information lookup) before DNS and Google. Agents need the same evolution: reliable address resolution for known agents, and semantic discovery for finding the right agent for a novel task. The 2030 Almanac should accept a natural-language capability description and return agents ranked by match, reputation, price, and availability—and should support real-time capability negotiation between a requesting and a candidate agent. The scale at which the Almanac already operates—36,000+ agents as of Q1 2026 ([Kantor, 2026](#))—makes this capability urgent.

6.3 From Chat Protocol to Agent Lingua Franca

Current state. The Agent Chat Protocol is the native communication standard. A2A and MCP are supported only via external adapters, not platform-integrated.

2030 state. A multi-protocol gateway bridging Chat Protocol, A2A ([Linux Foundation and Google, 2026](#)), and MCP ([Anthropic, 2025](#)), enabling Agentverse agents to participate in any compliant multi-agent system.

The tcp/ip parallel. The internet protocol landscape of the 1980s was fragmented; TCP/IP won by becoming the translation layer. By 2030, either one protocol dominates agent communication, or a gateway plays the TCP/IP role. The Agent Chat Protocol’s typed, schema-validated semantics are compatible with both A2A and MCP; the convergence may be more about shared *semantics*—a common ontology for expressing capabilities, tasks, and results—than about wire format. Agentverse is uniquely positioned to build this gateway, as it has already committed to both A2A adapter support and the most comprehensive MCP integration among decentralised platforms.

6.4 From Hosting to Agent Kubernetes

Current state. Each agent runs as a single instance with no health monitoring, no scaling, no rolling deployments, and no multi-region distribution.

2030 state. Agents deployed as scalable units with desired replica count, health probes, rolling update strategy, resource limits, and geographic placement constraints.

The dependency. Horizontal scaling requires that agent state be externalised. An agent that stores state in-process (even via `ctx.storage`, which is instance-local) breaks when replicated. This makes Layer 2 (Agent Memory) a prerequisite for Layer 1 (Agent Runtime) scaling—a dependency that must be designed into the architecture from the outset.

6.5 From Token Payments to Agent Economic Primitives

Current state. The Payment Protocol supports Stripe, Skyfire USDC, and ASI/FET on-chain payments. This is operational—agents can charge for services and pay each other.

2030 state. Multi-dimensional reputation, SLA-backed service agreements, resource lending, insurance pools, and dynamic price discovery.

The mechanism design framing. We use *agent economic primitives* deliberately to avoid conflating this with token speculation. Each primitive solves a specific information or commitment problem in agent markets: reputation reduces information asymmetry; SLA contracts enable credible commitment; insurance pools enable risk transfer; dynamic pricing enables allocative efficiency. These are mechanism design problems (Myerson, 2007) with known theoretical solutions—the engineering challenge is implementing them on a programmable blockchain at agent scale. The emergence of KYA (Know Your Agent) frameworks (ChainUp, 2026) and dedicated agent marketplaces (Coinbase / Agentic.Market, 2026) suggests the ecosystem is beginning to organise around exactly these needs.

7 Discussion

7.1 Competitive Positioning

Table 9 reveals a structural complementarity: hyperscaler platforms are strong on operational depth and weak on agent-native properties; Agentverse is the inverse. The competitive risk for Agentverse is not that hyperscalers build better agent hosting but that they integrate A2A so deeply into enterprise deployments that developers never feel the need to register on an open platform. The multi-protocol gateway (Section 6.3) is the mitigation: if an Agentverse agent can be discovered and called from any A2A client, the platform’s openness becomes a feature rather than an island.

It is worth noting the pace of hyperscaler investment: Amazon Bedrock AgentCore reached general availability in October 2025; Microsoft launched Entra Agent ID into public preview at Ignite 2025 and announced expanded Agent 365 capabilities at RSAC 2026; Google consolidated Vertex AI and Agentspace into a unified Gemini Enterprise Agent Platform at Cloud Next 2026. The window in which Agentverse can close the managed-service gap while retaining its decentralised identity advantage is real but not indefinite.

Table 9: Comparative capabilities of agent platforms, 2026 vs. projected 2030. ★★★★★ = excellent; ★○○○○ = minimal.

Dimension	Agentverse 2026	AWS Bedrock AgentCore	Azure AI Foundry	Google Gemini Ent.	Agentverse 2030
Compute	Hosted Python, single- instance	Serverless, managed	Managed, scal- able	Managed, scal- able	Wasm, multi- lang, auto-scale
Identity	Crypto keys	AWS IAM	Entra Agent ID	Google IAM	DIDs + VCs
Memory	ctx.storage	S3/DynamoDB	Cosmos DB	Firestore	Memory Cloud (4-tier)
Communication	Chat Protocol	REST/Bedrock API	REST/A2A	A2A/REST	Multi-protocol gateway
Discovery	Almanac	Bedrock Reg- istry	Copilot Studio	Agentspace	Semantic Agent DNS
Observability	Log polling	CloudWatch/X- Ray	Azure Monitor	Cloud Trace	OTel + cost dashboards
Security	Account-level	IAM/VPC/KMS	Entra + Azure Sec	Google IAM + VPC	Capability- based, Wasm
Payments	ASI + Stripe + USDC	AWS Billing	Azure Credits	GCP Billing	Agent econ. primitives
Decentralisation	★★★★★	★○○○○	★○○○○	★○○○○	★★★★★
Enterprise	★★○○○	★★★★★	★★★★★	★★★★○	★★★★○

7.2 Open Research Problems

Agent identity portability. Can a DID anchored on the ASI Chain be resolved and verified on AWS without that platform running a full ASI Chain node? This is a cross-chain identity resolution problem analogous to federated identity in Web2.

Memory privacy in multi-agent systems. If multiple agents share a memory space, what privacy guarantees can be made about information contributed by one agent from access by another? Federated learning approaches exist but their application to fine-grained agent memory is not well studied.

Agent coordination theory. The group coordination primitives we propose (consensus, delegation, quorum) require a theory of multi-agent coordination that goes beyond request-response. Game-theoretic models provide foundations; the specific dynamics of heterogeneous, incentive-driven agent networks require further theoretical development.

Verification of agent behaviour. How do we provide formal or probabilistic guarantees about what a deployed agent will and will not do, when the agent includes non-deterministic LLM components? Testable behavioural contracts and simulation-based evaluation (Layer 5) offer partial solutions.

Economic stability of agent markets. The economic primitives we propose must be incentive-compatible. Mechanism design provides the theoretical tools (Myerson, 2007); their application to autonomous agent markets with heterogeneous participants is an open problem.

7.3 Regulatory Considerations

The EU AI Act (high-risk provisions in force from August 2, 2026) classifies AI systems by risk level. Autonomous agents taking consequential actions—financial transactions, medical decisions, legal submissions—are likely to be classified as high-risk, triggering requirements for human oversight, auditability, and conformity assessment. The architecture we propose anticipates these requirements: the audit trail (Layer 5), human escalation protocol (Layer 3), capability-based permissions (Layer 1), and KYA registry (Layer 6) collectively provide the technical foundations for compliance. We argue for a mandatory baseline—every hosted agent should have an immutable action log and a human escalation path—with optional higher compliance tiers for regulated industries.

7.4 Limitations

Our analysis has three principal limitations. First, it is grounded in a single platform at a specific time; some gaps may not generalise to platforms with different architectural choices. Second, the 2030 projections are conditional on current trajectories and could be materially altered by technical breakthroughs or regulatory intervention. Third, the economic primitives in Sections 5.7 and 6.5 are proposed without formal analysis of their incentive properties; rigorous mechanism design treatment of each would require dedicated theoretical work.

8 Conclusion

We have presented a systematic analysis of the Agentverse platform as a case study in agent-native cloud infrastructure. Our empirical audit of 204 API endpoints reveals 52 capability gaps across eight categories—from agent memory and observability to security, communication, data services, economic primitives, and enterprise-grade scaling. These gaps are not failures of vision but the natural condition of pioneering infrastructure: foundations take time, and Agentverse has laid important ones.

Our seven-layer Agent Cloud Stack provides a reference architecture for closing these gaps by 2030. Each layer is grounded in specific identified gaps and informed by the evolution of general-purpose cloud computing as a reference model. The architecture preserves and extends the agent-native properties—cryptographic identity, open protocols, decentralised substrate—that distinguish Agentverse from its hyperscaler competitors, while adding the operational depth (memory, observability, security, enterprise services) that production deployments require.

The five critical evolution paths provide a concrete roadmap: from `ctx.storage` to Agent Memory Cloud; from keyword Almanac to Semantic Agent DNS; from single-protocol messaging to multi-protocol lingua franca; from single-instance hosting to agent-native orchestration; and from token payments to a full suite of agent economic primitives. Each path is a discrete engineering investment with a clear value proposition.

The broader significance lies in what this infrastructure enables. Hussein et al. (2025) project more than 100× growth in the global agent population between 2026 and 2036.

The infrastructure that governs how those agents interact—how they discover each other, remember their history, communicate across platforms, verify each other’s identity, and transact economically—will shape outcomes in commerce, science, and governance far beyond the scope of any individual agent application. Agentverse, uniquely among current platforms, is architecturally positioned to provide that infrastructure in an open, decentralised, and composable form. The distance between where it is today and where it needs to be by 2030 is large but well-defined. This paper is a map of that distance.

Acknowledgements. The authors thank Humayun Sheikh (Fetch.ai / ASI Alliance) for discussions on the Agentverse platform roadmap and the agentic web vision. This work was carried out independently; any errors or omissions are the authors’ own.

References

- AgentOps. AgentOps: AI agent monitoring. <https://agentops.ai>, 2026.
- Amazon Web Services. Amazon Bedrock AgentCore. <https://aws.amazon.com/bedrock/agentcore/>, 2025. Platform GA: October 13, 2025. Policy component GA: March 3, 2026. Evaluations component GA: March 31, 2026.
- Anthropic. Model context protocol specification (2025-11-25). <https://modelcontextprotocol.io/specification/2025-11-25>, 2025. MCP uses date-string versioning. Latest spec as of April 2026: 2025-11-25 (November 25, 2025). 110M+ monthly downloads.
- ASI Alliance. ASI Chain: AI-native blockchain infrastructure. <https://superintelligence.io>, 2026. DevNet beta: October 2025; TestNet: Q2 2026; Mainnet target: late 2026 / early 2027. Based on F1R3FLY blockDAG architecture.
- Bessemer Venture Partners. Securing AI agents: The defining cybersecurity challenge of 2026. <https://www.bvp.com/atlas>, 2026.
- ChainUp. Know your agent (KYA): The 2026 shift in AI & crypto. <https://chainup.com/blog>, 2026.
- Alan Chan, Kevin Wei, Sihao Huang, Nitarshan Rajkumar, Elija Perrier, Seth Lazar, Gillian K. Hadfield, and Markus Anderljung. Infrastructure for AI agents. *Transactions on Machine Learning Research*, 2025. arXiv:2501.10114.
- Coinbase / Agentic.Market. An app store for AI agents. <https://banklesstimes.com>, 2026. April 2026.
- EPH4 and WhatIsWeb4. Web 4.0: Agentic AI and the symbiotic Web. <https://eph4.ai>, 2026.
- Fetch.ai. Fetch.ai, SingularityNET, and Ocean Protocol unite to create the Artificial Superintelligence Alliance. <https://fetch.ai/blog>, 2024. Token merger finalized July 15, 2024. Ocean Protocol subsequently withdrew October 9, 2025.

- Fetch.ai. Fetch.ai agent payment system. <https://fetch.ai>, 2026. Includes Skyfire USDC on Base (partnership confirmed BusinessWire June 26, 2025 KYAPay launch), Stripe, and ASI on-chain payments.
- Sandro Rodriguez Garzon et al. AI agents with decentralized identifiers and verifiable credentials. arXiv preprint arXiv:2511.02841, 2025. Accepted ICAART 2026 (18th International Conference on Agents and Artificial Intelligence). v2: December 15, 2025.
- Google. Gemini Enterprise Agent Platform (Google Cloud Next 2026). <https://cloud.google.com/blog/topics/google-cloud-next/welcome-to-google-cloud-next26>, 2026. Announced April 22, 2026. Vertex AI rebranded as Gemini Enterprise Agent Platform; Agentspace absorbed into unified Gemini Enterprise product.
- Gillian K. Hadfield and Andrew Koh. An economy of AI agents. arXiv preprint arXiv:2509.01063, 2025. Submitted September 1, 2025. Hadfield: Johns Hopkins; Koh: MIT.
- Ahmed Refaey Hussein et al. When intelligence overloads infrastructure: A forecast model for AI-driven bottlenecks. arXiv preprint arXiv:2511.07265, 2025. Projects AI agent population to grow 100× between 2026 and 2036, reaching 50B–5T instances.
- Michael Kantor. State of AI agents — March 2026. DEV Community, <https://dev.to>, 2026. Cross-platform snapshot, March 2026. Reports 36,338 agents on AgentVerse (Fetch.ai), representing 34.8% share of catalogued agents. Fetch.ai Nov 2025 press release cited 2M+ agents in broader ecosystem.
- LangChain. LangSmith: Observability for LLM applications. <https://smith.langchain.com>, 2026.
- Langfuse. Langfuse: Open-source LLM engineering platform. <https://langfuse.com>, 2026.
- Linux Foundation and Google. Agent2Agent (A2A) protocol v1.0. <https://a2a-protocol.org>, 2026. v1.0 announced April 9, 2026. 150+ supporting organisations. Linux Foundation governance. Founding TSC partners: AWS, Cisco, Google, IBM Research, Microsoft, Salesforce, SAP, ServiceNow. Adds Signed Agent Cards and formal AP2 extension.
- MCP Core Maintainers. The 2026 MCP roadmap. <https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/>, 2026. March 2026. Explicitly states: “MCP’s current spec release came out in November 2025. We haven’t cut a new version since”.
- Mem0. State of AI agent memory 2026. <https://mem0.ai/blog/state-of-ai-agent-memory-2026>, 2026. Reports 21 frameworks, 19 vector stores, 3 hosting models: managed cloud, open-source self-hosted, and local MCP.
- Microsoft. Microsoft Entra Agent ID. <https://learn.microsoft.com/en-us/entra/>, 2025. First announced May 2025; public preview expanded at Microsoft Ignite November 2025; governance capabilities extended at RSAC 2026. Agent 365 GA May 1, 2026.

- Microsoft Open Source. Introducing the Agent Governance Toolkit. <https://github.com/microsoft/agent-governance-toolkit>, 2026. Open-sourced April 2, 2026. MIT licence. Covers all 10 OWASP Agentic Top 10 risks. Available in Python, TypeScript, Rust, Go, and .NET.
- MolTrust. Trust infrastructure for AI agents. <https://moltrust.ch>, 2026. W3C Verifiable Credentials for AI agents; ERC-8004 on-chain registration; Base blockchain anchoring; MCP server available.
- Roger B. Myerson. Mechanism design. Nobel Prize Lecture, Royal Swedish Academy of Sciences, 2007. <https://www.nobelprize.org/prizes/economic-sciences/2007/>.
- Ocean Protocol Foundation. The ASI Alliance from Ocean’s perspective. <https://blog.oceanprotocol.com>, 2025. Ocean Protocol Foundation announced withdrawal effective October 9–10, 2025. Subsequent litigation over 263M FET.
- OWASP Foundation. OWASP top 10 for agentic applications. <https://genai.owasp.org>, 2025. Published December 9, 2025.
- W3C. Decentralized identifiers (DIDs) v1.0. <https://www.w3.org/TR/did-core/>, 2022.
- Zep AI. Zep: A temporal knowledge graph architecture for agent memory. arXiv preprint arXiv:2501.13956, 2025. Achieves 63.8% accuracy on LongMemEval (GPT-4o) vs 49.0% for flat vector stores.